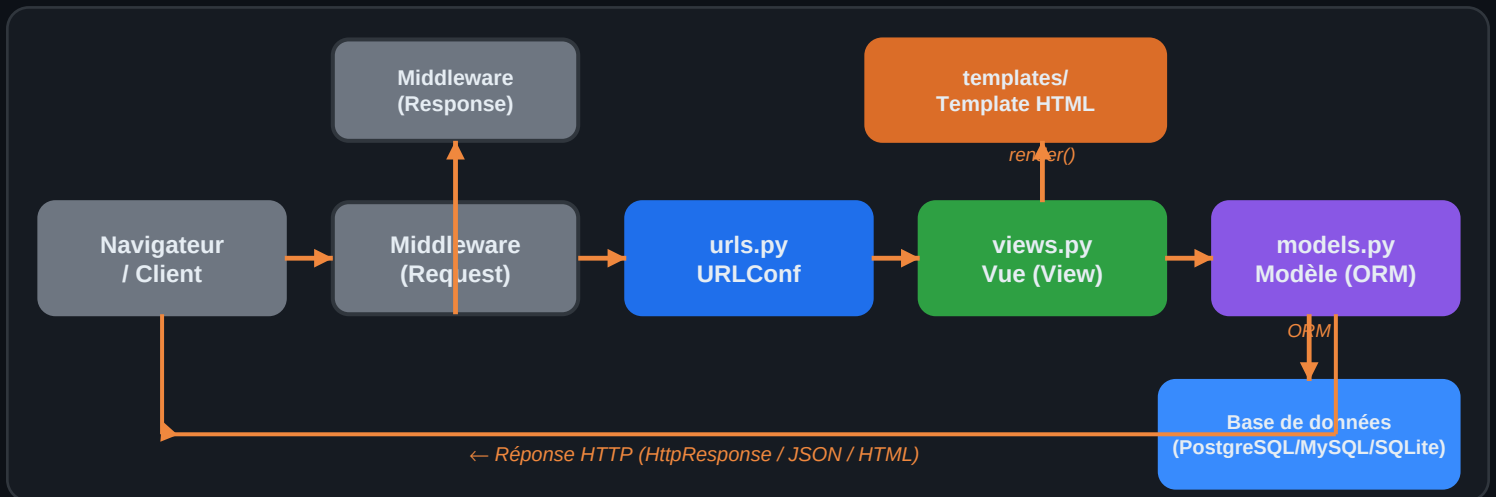


Architecture Django — Schéma Détaillé

Modèle MVT (Model – View – Template) | Page 1/2

① Cycle Requête / Réponse



② Les Composants Clés de Django

urls.py	views.py	models.py	templates/	admin.py
• URLconf • Routage des URL • Associe URL → Vue • <code>path()</code> / <code>re_path()</code> • <code>include()</code> pour sous-modules	• Logique métier • FBV ou CBV • Récupère données • Retourne réponse • <code>render()</code> / <code>redirect()</code> • <code>JsonResponse()</code>	• Définit les tables • ORM (QuerySet) • Héritage <code>models.Model</code> • Champs : <code>CharField</code>, <code>IntegerField</code>... • Migrations auto	• Fichiers HTML • Moteur Jinja2-like • <code>{{ variable }}</code> • <code>{% tag %}</code> • Héritage templates • <code>{% block %}</code> / <code>extends</code>	• Interface <code>/admin/</code> • CRUD automatique • <code>register(Model)</code> • Personnalisable • Filtres & recherche • Actions groupées

③ Structure d'un Projet Django

```
mon_projet/
manage.py      ← Commandes (runserver, migrate, createsuperuser...)
mon_projet/    ← Dossier de configuration principal
settings.py    ← Config : BDD, apps, middleware, static...
urls.py        ← URL racine du projet
wsgi.py / asgi.py ← Point d'entrée serveur (WSGI/ASGI)
mon_app/       ← Application métier (ex: comptes, articles)
models.py      ← Modèles de données
views.py       ← Logique de traitement
urls.py        ← URL de l'application
admin.py       ← Enregistrement admin
migrations/    ← Historique des changements BDD
templates/     ← Fichiers HTML
static/        ← CSS, JS, images
```

Django — Concepts Avancés & Outils

ORM · Middleware · Formulaires · Signaux · Commandes | Page 2/2

④ L'ORM Django (Object-Relational Mapping)

Définir un Modèle

```
class Article(models.Model):
    titre = models.CharField(max_length=100)
    contenu = models.TextField()
    date = models.DateTimeField(auto_now=True)
    auteur = models.ForeignKey(
        User, on_delete=models.CASCADE)

Champs courants :
CharField, TextField, IntegerField,
DateField, BooleanField, ImageField,
ForeignKey, ManyToManyField, OneToOneField
```

QuerySet (Lecture)

```
# Tous les objets
Article.objects.all()

# Filtrage
Article.objects.filter(auteur=user)
Article.objects.exclude(publie=False)

# Un seul objet
Article.objects.get(pk=1)

# Tri & limite
Article.objects.order_by('-date')[:5]
```

Écriture & Migrations

```
# Créer
a = Article(titre='Hello')
a.save() # ou Article.objects.create(...)

# Modifier
a.titre = 'Nouveau titre'
a.save()

# Supprimer
a.delete()
```

Migrations :
makemigrations → migrate

⑤ Middleware · Formulaires · Signaux

Middleware

Couche entre requête et vue.

Ordre d'exécution :

- Requête → du haut vers le bas
- Réponse ← du bas vers le haut

Middlewares courants :

- SecurityMiddleware
- SessionMiddleware
- AuthenticationMiddleware
- CsrfViewMiddleware
- MessageMiddleware

Formulaires (forms.py)

```
class LoginForm(forms.Form):
    email = forms.EmailField()
    password = forms.CharField(
        widget=forms.PasswordInput)

ModelForm (auto depuis modèle) :
class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ['titre', 'contenu']

Validation : form.is_valid()
Données : form.cleaned_data
```

Signaux (signals.py)

Réagir à des événements.

Exemple post_save :

```
@receiver(post_save, sender=User)
def on_user_created(sender, instance, created, **kw):
    if created:
        send_welcome_email(instance)
```

Signaux Django :

- pre_save / post_save
- pre_delete / post_delete
- m2m_changed

⑥ settings.py · Fichiers Statiques · Commandes manage.py

settings.py — Configuration

INSTALLED_APPS → apps activées
DATABASES → connexion BDD
MIDDLEWARE → liste middlewares
TEMPLATES → moteur templates
STATIC_URL → chemin statiques
MEDIA_ROOT → fichiers uploadés
AUTH_USER_MODEL → modèle utilisateur
SECRET_KEY → clé cryptographique
DEBUG → mode développement

Fichiers Statiques & Médias

static/ → CSS, JS, images du projet
media/ → fichiers uploadés par users

settings.py :

```
STATIC_URL = '/static/'
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

collectstatic → prod (WhiteNoise/nginx)
{% load static %} dans templates

Commandes manage.py

runserver → lancer le serveur
makemigrations → créer migrations
migrate → appliquer migrations
createsuperuser → admin compte
shell → console Python
collectstatic → regrouper statiques
startapp <nom> → nouvelle app
test → lancer les tests
dbshell → console base données

⑦ Authentification · Sécurité intégrée

Authentification

- login() / logout() / authenticate()
- @login_required (décorateur)
- LoginView, LogoutView (CBV)
- Permission : @permission_required
- Groupes & permissions par modèle

Protection CSRF

- Token auto dans les formulaires
- {% csrf_token %} dans HTML
- Middleware CsrfViewMiddleware
- @csrf_exempt si API externe
- Protection XSS via échappement auto

Sessions & Cookies

- request.session[clé] = valeur
- Stockage : BDD / Cache / Cookie
- SESSION_COOKIE_AGE (durée)
- SESSION_COOKIE_SECURE (HTTPS)
- Hashage md5 : PBKDF2 par défaut